PolicySpace Manual

PolicySpace is a game about economies and policy. It is being built one system at a time, and so far three systems exist: **the world**, **the people in it**, and **the businesses that produce**. A *setting* describes a world: its goods, its map with a currency on every hex, its natural resources, the prices its markets post, the groups of people, called **POPs**, who live on it, the **technologies** (the recipes factories run), and the **businesses** that own deposits and factories. The engine loads the setting and runs turns in which the POPs earn, trade, consume and grow, and the businesses, each steered by a small decision program, buy, produce and sell. A developer tool lets you watch all of it happen. There is no player-facing game yet, so this manual is written for **setting authors** and developers.

This manual grows as each part of the project is completed.

Contents

Project overview **2**

Writing a setting **3**

- The folder layout . . . 3
- Naming rules . . . 4
- Goods . . . 4
- The map . . . 5
- Natural resources . . . 7
- Markets . . . 9
- POP types . . . 11
  - How POPs grow . . . 12
- POPs . . . 14
- Technologies . . . 16
  - What the numbers mean . . . 17
- Businesses . . . 20
  - How the arithmetic rule decides . . . 21
- Factories . . . 23
- Running a setting . . . 25
  - How POPs trade . . . 26
  - How businesses trade and produce . . . 28
  - How a POP's size changes . . . 30
  - Turn reports . . . 30
- Checking a setting . . . 32
- The developer UI . . . 34
  - Reading the map . . . 34

Inspecting a hex 34
Stepping turns 38
The world panel 38
The author's loop 38
Not available yet 39

Technical documentation **39**

Project overview

The project has three parts. The **engine** library (Rust) holds the rules and runs the world. The **developer UI** program (Rust) loads settings and shows them. The **Python package**, `policyspace`, holds the **deciders**: the programs that choose, every turn, what each business does. The engine never depends on the developer UI, and it never imports the Python code: it starts a decider as a separate process and exchanges one message per turn with it, which is why deciders can be written in Python. Details are in docs/workspace.md and docs/python.md, and the way names become internal IDs is explained in docs/ids.md.

Setting up. Run these two commands once, from the project's root folder (Python 3.12 or later; the first install needs network access):

```
python -m venv .venv
.venv/Scripts/python -m pip install -r requirements.txt
```

They create a virtual environment holding the Python package, the test runner and Jupyter. The developer UI needs it to load any setting that has businesses, the sample included, and the tests need it too. Without it, loading such a setting fails with a message that names where the environment was expected and repeats the two commands above. A setting with no businesses needs no Python at all.

Two commands matter day to day, both run from the project's root folder:

- `cargo run -p policyspace-dev-ui -- <folder>` opens the developer UI on a setting (see The developer UI).
- `.venv/Scripts/python tools/check.py` runs four checks: formatting, lints, the Rust tests and the Python tests. Run it with the environment's Python, as written, because the Python tests run under whichever Python starts the script; if it complains that pytest is missing, rerun the install command.

Two environment variables are there for developers: `POLICYSPACE_PYTHON` points the developer UI and the tests at another Python interpreter, and `POLICYSPACE_DECIDER_TIME_LIMIT` sets how many seconds the engine waits for a decider's answer each turn (normally 5; `0` means wait forever, which is what you want when stepping through a decider in a debugger).

See docs/workspace.md.

Writing a setting

A **setting** is a folder of plain-text TOML files that describes a world: what goods exist, what the map looks like and which good each hex uses as its currency, where natural resources lie, what the markets charge, which groups of people live where, which recipes factories can run, and which businesses own the deposits and factories. The engine has no built-in content, so everything comes from the setting. A complete example lives in `settings/sample/`; copy it to start your own.

For the technical side of the format, see docs/setting-loader.md.

The folder layout

```
my_setting/
  setting.toml      the manifest
  rules/            things that never change during play (goods, resource
                    types, POP types, technologies)
  start/            what turn 0 looks like (the map and its currencies,
                    the deposits, the markets,
                    the POPs, the businesses and their factories)
```

The manifest, `setting.toml`, has these keys:

Key	Required	Meaning
`name`	yes	The setting's name
`description`	no	A sentence about it
`format_version`	yes	Always `1` for now

A minimal manifest:

```toml
name = "Sample Setting"
description = "The project's sample setting."
format_version = 1
```

Unknown keys are errors everywhere in a setting, not just in the manifest. A misspelled key is reported instead of silently ignored.

Files and folders.

- Organize files however you like, nested to any depth. Files are read in sorted path order.

- What a file defines comes from its section headers (such as `[[good]]`), not from its file name.
- `README.md` files are the place for notes. Hidden files, `Thumbs.db` and `desktop.ini` are ignored. Any other file that does not end in `.toml` is an error.
- Each section type belongs in one of the two folders: `[[good]]`, `[[resource]]`, `[[pop_type]]` and `[[technology]]` in `rules/`; `[[hex]]`, `[[deposit]]`, `[[market]]`, `[[pop]]`, `[[business]]` and `[[factory]]` in `start/`. Putting one in the wrong folder is an error.

Naming rules

Names of goods, resource types, POP types, technologies and businesses use lowercase letters, digits and underscores, and must start with a letter: `copper_ore` and `grain2` are fine; `Copper`, `2grain` and `copper-ore` are not. Each kind of thing has its own list of names, so a resource type, a POP type, a technology or a business may share a name with a good. Hexes, deposits, POPs and factories have no names.

Goods

A **good** is anything an actor can own. The engine has no built-in goods and no built-in categories such as labor, money or capital: the setting defines them all.

Define goods with `[[good]]` sections in `rules/`, in any file at any depth, as many per file as you like. Goods from all files are merged. A setting with no goods is valid. A `[[good]]` in `start/` is an error.

Key	Required	Meaning
`name`	yes	Lowercase snake_case, unique across the whole setting
`perishes_after`	no	A whole number, 1 or more. Leave it out and the good never perishes.
`movable`	no	`true` (the default) or `false`. **It has no effect yet**; movement comes in a later stage.

`perishes_after` counts the turn a unit appears in. A unit appearing in turn 5 with `perishes_after = 1` is gone when turn 5 ends; with `perishes_after = 3` it is gone at the end of turn 7.

```toml
# Labor: gone at the end of the turn, and tied to where it is.
[[good]]
name = "unskilled_labor"
perishes_after = 1
movable = false
```

```
# Only a name is needed: durable and movable.
[[good]]
name = "copper"
```

The sample setting defines seven goods:

Name	perishes_after	movable	Role
unskilled_labor	1	false	Labor: what the sample's POPs receive every turn
unimproved_copper	1	false	What copper deposits yield
unimproved_grain	1	false	What arable land yields
copper	never	true	A durable, movable good that artisans buy
grain	3	true	A perishable, movable good that POPs eat
silver	never	true	The currency every sample hex uses
tools	never	true	The capital good the sample's factories are made of, and what its toolmaker makes

More detail: docs/goods.md.

The map

The **map** is the set of hex tiles in the setting's starting state. For now a hex has only a position and a currency; terrain and other attributes come later. The map goes in `start/`, which describes what turn 0 looks like. A `[[hex]]` in `rules/` is an error.

Each `[[hex]]` has exactly three required keys: the whole numbers `q` and `r`, and `currency`, the name of a good (explained below). Any other key (such as `terrain`) is an error. Hexes have no names: the coordinates are the identity, and two hexes at the same coordinates is an error that names both places. Hexes can sit in any file at any depth and are merged.

```
[[hex]]
q = 0
r = 0
currency = "silver"

[[hex]]
q = 1
r = 0
currency = "silver"
```

Coordinates and orientation. The engine fixes the orientation; a setting cannot change it. Hexes are *pointy-top*. Each value of `r` is one straight east-west row; `q` increases eastward and `r` increases southward. So `(1, 0)` is the east neighbor of `(0, 0)`, and the row `r = 1` lies south of the row `r = 0`. Values may be negative, up to about plus or minus 2.1 billion.

Shape. There are no shape rules. Holes, islands and negative coordinates are all fine, and a setting with no hexes is valid. Neighbors, distance and wrapping around the edges do not exist yet; wrapping is planned for later as an optional per-setting value.

Currency. Every hex names the good that prices on that hex are quoted in: its market posts prices in it, and the POPs there are paid in it when they sell and pay with it when they buy. Any good the setting defines can be a currency. The engine has no idea of "money", so a perishable or unmovable good is allowed, and if you name labor as the currency, that is exactly what you get. Different hexes may name different goods, and there is no map-wide default: every hex writes its own.

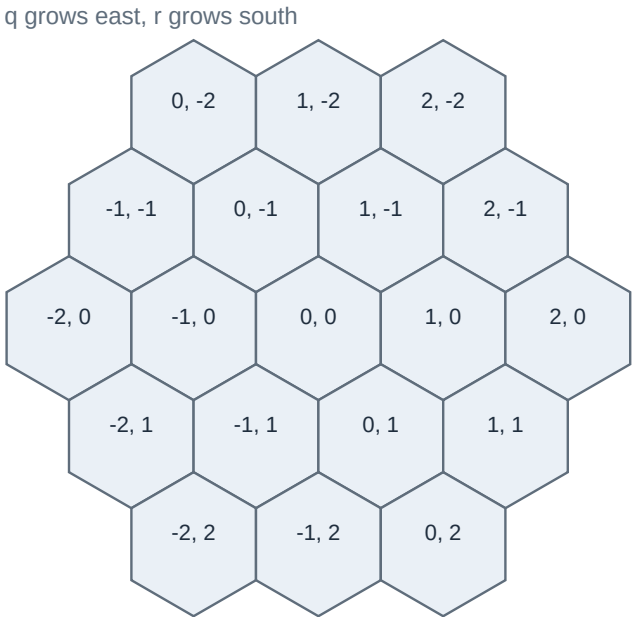
```
[[hex]]
q = 0
r = 0
currency = "coin"

[[hex]]
q = 1
r = 0
currency = "shell"     # the next hex along trades in a different good
```

A hex without a `currency` key, one whose value is not text, and one naming a good that does not exist (including a name in the wrong case, such as `"Silver"`) are each an error with file, line and column. Like the other errors that refer to names, the unknown-good one is not shown while part of the setting could not be read. Because the key is required, a setting written before currencies existed will not load until every hex has one.

The sample map is a hexagon of radius 2 around `(0, 0)`: 19 hexes, every one with `currency = "silver"`. The diagram below shows it, with each hex labelled `q, r`.

More detail: docs/map.md for the map and docs/currency.md for currencies.



Natural resources

Natural resources come in two parts: **resource types** say what kinds of resource exist and what one deposit produces, and **deposits** put them on the map. Each deposit produces its type's output every turn (see Running a setting). A deposit can be owned by one business; the ownership is written on the business, not on the deposit (see Businesses). An owned deposit produces into its owner's stockpile, an unowned one into the hex's stockpile of unowned goods. There is no stock or depletion.

Resource types are `[[resource]]` sections in `rules/` (in `start/` it is an error), in any file at any depth, merged across files. Three keys are required:

Key	Meaning
`name`	Lowercase snake_case, unique among resource types. It may match a good's name.
`produces`	The name of a good the setting defines
`output`	How much of that good one deposit produces per turn: a whole number or decimal, greater than 0

Each type produces exactly one good, but several types may produce the same one. By convention the produced good is unmovable and perishes after one turn (like the sample's `unimproved_*` goods), but the engine does not require that.

Deposits are `[[deposit]]` sections in `start/` (in `rules/` it is an error). Keys:

Key	Meaning
`resource`	The name of a resource type
`q`, `r`	A hex that exists on the map
`count`	Optional, 1 to 1,000,000, default 1

`count = 3` makes three separate deposits, each of which can have its own owner, so the total on a hex is `count` times `output`. A hex can hold several resource types, but each type may appear only once per hex: raise `count` instead.

```
# rules/resources.toml
[[resource]]
name = "copper_ore"
produces = "unimproved_copper"
output = 2.5

# start/deposits.toml
[[deposit]]
resource = "copper_ore"
q = 1
r = -1
count = 2

[[deposit]]
resource = "copper_ore"
q = -2
r = 1
```

A resource's `produces`, a deposit's `resource`, and a deposit's hex are checked after all files are read, so the target can be in any file. These reference errors are not shown if part of the setting could not be read (see Checking a setting).

The sample setting has two resource types:

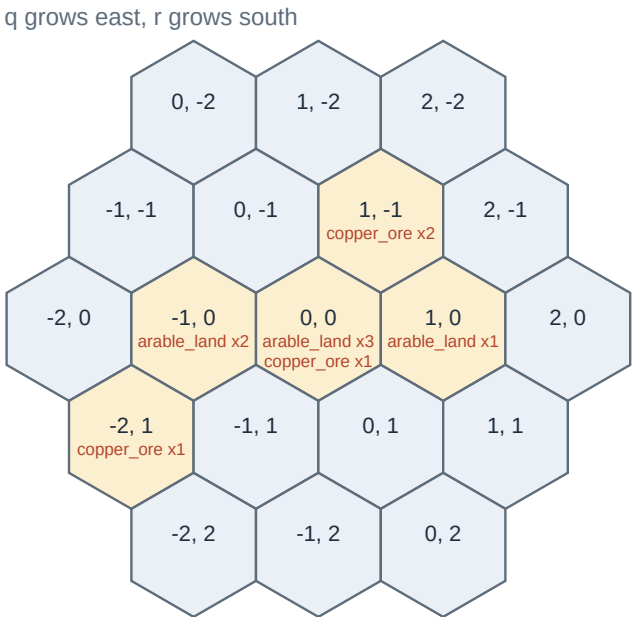
Name	Produces	Output per deposit
`copper_ore`	`unimproved_copper`	2.5
`arable_land`	`unimproved_grain`	4

and six deposit entries that place 10 deposits:

Hex `(q, r)`	Resource	Count
`(0, 0)`	`arable_land`	3
`(0, 0)`	`copper_ore`	1

Hex (q, r)	Resource	Count
(-1, 0)	arable_land	2
(1, 0)	arable_land	1
(1, -1)	copper_ore	2
(-2, 1)	copper_ore	1

Hex (0, 0) holds two resource types at once, and three of its four deposits are owned by the sample's businesses (see Businesses); every other deposit is unowned. In the diagram, hexes with deposits are shaded.



Planned for later: stock, depletion and regeneration as optional data on resource types, so settings written now stay valid.

More detail: docs/resources.md, with docs/map.md for coordinates and docs/goods.md for goods.

Markets

A **market** is where the POPs on a hex buy and sell. For now it is a stand-in: it posts fixed prices and buys or sells any amount at them. It holds no stock of its own and never runs out: goods sold to it vanish, goods bought from it appear from nothing, and the currency paid or received does the same. A later stage replaces it with a real market whose prices move with supply and demand. Until then, the prices you write are what everyone trades at.

Markets are [[market]] sections in start/ (in rules/ it is an error, since prices are state

that a live market will change). Keys:

Key	Meaning
`q`, `r`	A hex that exists on the map
`prices`	A table of `good name = price`. Each price is a number greater than 0: how many units of the hex's currency one unit of the good costs

The rules that follow from this:

- **A good is traded on a hex only if that hex's market lists a price for it.** A good with no price there is neither bought nor sold there, however much anyone wants it. A good may exist in `rules/` long before any hex trades it.
- A hex with no `[[market]]` at all is valid and trades nothing. So is a market with `prices = {}`.
- At most one market per hex.
- Prices are in the hex's currency (see The map). The currency itself always costs 1, so listing it in `prices` is an error. A good may be priced on a hex whose currency is a different good.
- Both TOML forms of the table work: inline, or a `[market.prices]` sub-table.

```
# Inline: grain and fish are traded here; copper, with no price, is not.
[[market]]
q = 0
r = 0
prices = { grain = 2, fish = 1.5 }

# The same thing written as a sub-table.
[[market]]
q = 1
r = 0

[market.prices]
grain = 3
fish = 1
```

How a trade works (what POPs do with it is in Running a setting): selling removes the good from the seller's stockpile and adds the price times the amount in currency; buying does the reverse. A bought good, and currency received from a sale, counts as appearing in the turn of the trade, so a perishable one starts its countdown then. When goods leave a stockpile, the units that would perish soonest go first.

Errors, each with file, line and column: a missing or bad `q` or `r`; coordinates with no hex; a second market on the same hex (the message names the first); a missing `prices` or one that is not a table; a price that is not a number or not greater than 0 (`inf` included); an unknown good in `prices`; the hex's own currency listed (`'silver' is the currency at (0, 0), so`

`it can't have a price`); and an unknown key. The unknown-good and currency checks are skipped while part of the setting could not be read.

The sample setting gives 18 of its 19 hexes a market, with prices in `silver`:

Hexes	unskilled_ labor	grain	copper	unimproved_ grain	unimproved_ copper	tools
the center `(0, 0)`	1	2	5	0.5	1	12
every other hex except `(2, -2)`	1	2	5	not traded	not traded	not traded
`(2, -2)`, the north-east corner	no market					

So the sample has one hex that trades every good, seventeen that trade some, and one that trades nothing. The center's prices are what the sample's technologies are balanced against: at these prices a farm and a mine clearly pay, and a toolmaker only just (see Technologies). The other hexes' prices are placeholders.

More detail: docs/market.md.

POP types

A **POP** is a group of identical people of one kind living on one hex, and a **POP type** describes one person of that kind: what they receive each turn, what they want to consume, and how their numbers grow. POP types are rules, so they go in `rules/` as `[[pop_type]]` sections (in `start/` it is an error). Mind the two headers: `[[pop_type]]` in `rules/` describes a person, and `[[pop]]` in `start/` places a group of them (see POPs).

Four keys, all required:

Key	Meaning
`name`	Lowercase snake_case, unique among POP types
`endowment`	`good name = amount` pairs: what one person receives at the start of every turn. Amounts are numbers greater than 0. May be empty (`{}`), for a type with no income
`utility`	One entry per good a person consumes, each with two optional numbers that default to 0: `subsistence`, the amount needed before anything else, and `share`, the good's weight when spending whatever is left. Each listed good needs at least one of them above 0, and at least one good needs a `share` above 0
`growth`	How a group's size responds to its prosperity: three values described under How POPs grow below

Things worth knowing:

- **Shares are weights, not percentages.** The engine scales them to add up to 1, so shares of `1`, `4` and `5` mean 10%, 40% and 50%. You never have to make them add up yourself.
- **A pure necessity** is a good with a `subsistence` and no `share`: a person buys it up to the subsistence amount and never more.
- **Leisure needs no special setting.** A good that is endowed *and* in the utility is kept and consumed: that is leisure. An endowed good that is not in the utility is sold entirely; a wanted good that is not endowed is bought.
- **What to endow.** The intended design gives people nothing but unskilled labor, which they sell and partly keep. Any good is allowed, the currency included, so the system can be tried out before production exists.
- **A pitfall.** The engine does not compare a good's subsistence with its endowment. For a good only the person produces, such as labor, a subsistence at or above the endowment can never be reached.
- Both TOML forms work: inline tables, or sub-tables such as `[pop_type.utility.grain]`.

How POPs grow

Each turn, after it has traded and consumed, a POP's size changes by a rate that depends on how far above or below subsistence it was. The measure is its **subsistence index**: how many extra subsistence bundles the POP's income could have bought. It is 0 when income exactly covers subsistence, 1 when income is twice the cost of subsistence, and negative below subsistence, down to −1 for a POP that has nothing. The `growth` table has three required keys:

Key	Meaning
`growth`	A list of one or more numbers giving the shape of growth above subsistence: with an index *u*, the rate is `c1·u + c2·u² + ….` One number gives a straight line; a negative second number makes growth peak and then fall away as the POP gets richer
`decline`	A number from 0 up to (not including) 1: how fast a POP with nothing shrinks each turn. Below subsistence the shrinking is in proportion, so a POP that gets half of what it needs shrinks by half of `decline`
`max_growth`	A number of at least 0: the most a POP can grow in one turn

The rate always stays between `−decline` and `max_growth`, and a POP is never removed: a starving POP loses the same fraction every turn but never reaches zero. `growth = [0]` with `decline = 0` freezes a type's size, which is useful for a setting that should stay put.

Two things about "below subsistence" matter here. A POP that cannot get a good it needs at all, because its hex does not sell it and it holds none, counts as having nothing and shrinks at the full `decline`, however rich it is otherwise. And a type whose subsistence goods cost nothing where it lives (or that has no subsistence at all) neither grows nor shrinks.

```
# Inline form. Farmers keep some labor as leisure and spend the rest on grain.
[[pop_type]]
name = "farmer"
endowment = { labor = 10 }
utility = { labor = { subsistence = 1, share = 1 }, grain = { subsistence = 2, share = 3 } }
growth = { growth = [0.06, -0.03], decline = 0.1, max_growth = 0.05 }

# Sub-table form. Fishers need one fish and want no more than that (a pure
# necessity), spend the rest on coin, and grow in a straight line.
[[pop_type]]
name = "fisher"

[pop_type.endowment]
fish = 2
labor = 8

[pop_type.utility.fish]
subsistence = 1

[pop_type.utility.coin]
share = 0.5

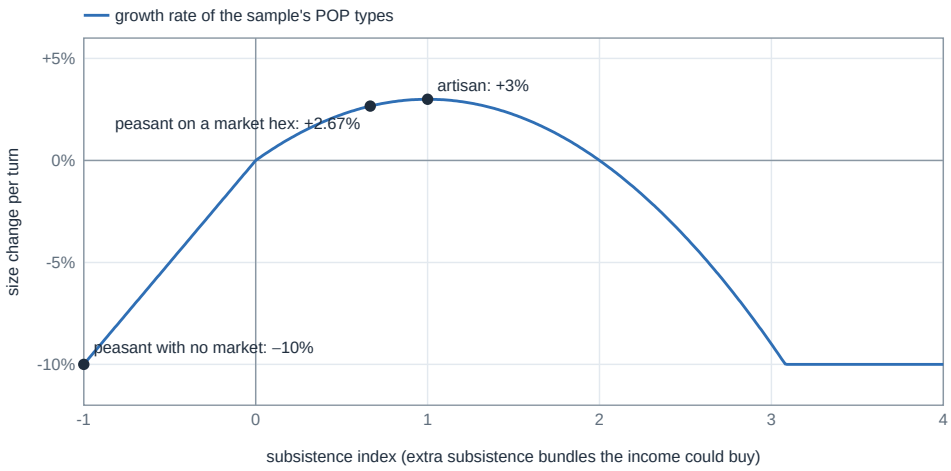
[pop_type.growth]
growth = [0.03]
decline = 0
max_growth = 1
```

Errors, each with file, line and column: a missing key; a bad or duplicate name; an `endowment`, `utility` or `growth` that is not a table; an endowment amount that is not a number above 0; a utility entry that is not a table; a negative or non-finite `subsistence` or `share`; a utility good with nothing above 0; a utility with no positive share; a `growth` list that is empty or holds something that is not a finite number; a `decline` outside 0 to 1; a negative `max_growth`; an unknown good; and an unknown key anywhere. Unknown-good errors are skipped while part of the setting could not be read. Because `growth` is required, a type written without it will not load until it is added.

The sample setting defines two types, both receiving 10 `unskilled_labor` per person per turn:

Type	Good	Subsistence	Share as written	Share used
`peasant`	`unskilled_labor` (leisure)	2	0.2	0.2
	`grain`	2	0.8	0.8
`artisan`	`unskilled_labor` (leisure)	1	1	0.1
	`grain`	2	4	0.4
	`copper`	0	5	0.5

Peasants keep some labor as leisure and spend mostly on grain; artisans keep less leisure and weight copper heavily. The artisan rows show weights being scaled to shares. Both types grow with `growth = [0.06, -0.03]`, `decline = 0.1` and `max_growth = 0.05`: growth peaks at 3% a turn for a POP whose income is twice its subsistence cost, falls back to zero at three times, and a POP with nothing shrinks 10% a turn. The chart shows that rule, with the three situations the sample's POPs are in (where those numbers come from is in Running a setting).



More detail: docs/pops.md for the keys and every error message, docs/growth.md for the growth rule, and docs/goods.md for the goods a type names.

POPs

A `[[pop]]` section in `start/` places a group of people of one type on one hex (in `rules/` it is an error, since a POP changes during play). Four keys, all required:

Key	Meaning
`type`	The name of a POP type from `rules/`
`q`, `r`	A hex that exists on the map, as for deposits and markets
`size`	The number of people at turn 0: a number greater than 0, decimals allowed

- **Size is a starting value, and it need not be whole.** A POP is an aggregate: what it receives and wants scales with its size, and every turn the size is multiplied by a growth rate, so after the first turn sizes are long decimals anyway. Nothing is rounded, in the engine or on screen.
- **At most one POP of each type per hex.** Two groups of the same type on one hex would behave as one group split in two, so a second `[[pop]]` with the same type and coordinates is an error that names the first. A hex may hold POPs of several types, or none.
- **POPs start with nothing.** There is no starting cash or goods. At the start of every turn,

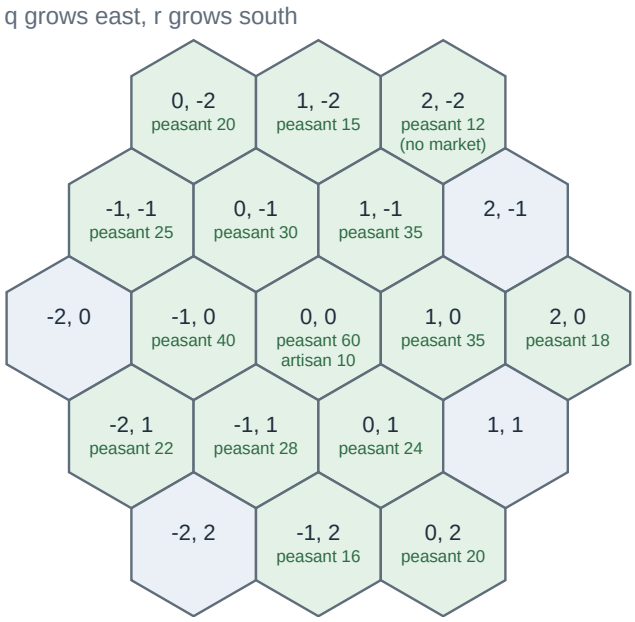
turn 0 included, each POP receives its type's endowment times its size, and it perishes like any other good: the sample's labor would be gone at the end of the turn it arrives if the POP did not sell or consume it, while an endowed durable good would pile up.
- POPs never receive deposit output. It goes to the business that owns the deposit or, if nobody does, to the hex's stockpile of unowned goods.

```
[[pop]]
type = "peasant"
q = 0
r = 0
size = 60

[[pop]]
type = "artisan"     # a second type on the same hex is fine
q = 0
r = 0
size = 2.5           # sizes need not be whole numbers
```

Errors, each with file, line and column: a missing `type`, `q`, `r` or `size`; a `type` that is not text; bad coordinates or coordinates with no hex; a `size` that is not a number above 0; a duplicate POP; an unknown POP type; an unknown key; and a `[[pop]]` in `rules/`. The unknown-type and no-hex errors are skipped while part of the setting could not be read.

The sample setting places 16 POPs, 410 people in all: a `peasant` POP on 15 of the 19 hexes, with sizes from 12 to 60, and an `artisan` POP of 10 at the center `(0, 0)`, the only hex that prices every good, so the center holds two POPs. The peasant POP of 12 on the north-east corner `(2, -2)` has no market, so it can never sell its labor or buy grain; it is there to show what happens to a POP that cannot trade. Four hexes have no POP: `(2, -1)`, `(-2, 0)`, `(1, 1)` and `(-2, 2)`. In the diagram, hexes with POPs are shaded.



More detail: docs/pops.md.

Technologies

A **technology** is a recipe that factories run: what a factory of that kind makes and from what, how much it can make in a turn, what a finished factory is made of, what building one takes, and how long it lasts. A technology is shared by every factory that uses it, the way a POP type is shared by POPs. Technologies are rules, so they go in `rules/` as `[[technology]]` sections (in `start/` it is an error). The factories themselves are placed in `start/` and belong to businesses (see Factories and Businesses).

Eight keys, all required:

Key	Meaning
`name`	Lowercase snake_case, unique among technologies
`inputs`	`good name = amount` pairs: what making one unit of output takes. May be empty (`{}`)
`output`	The name of the one good a factory of this kind makes
`capacity`	The most output one factory makes in one turn: a number greater than 0
`capital`	`good name = amount` pairs: the goods a finished factory is made of. At least one good
`construction`	`good name = amount` pairs: the goods used up to build one factory. May be empty (`{}`)
`construction_turns`	How many turns building a factory takes: a whole number, 1 or more

Key	Meaning
`lifetime`	How many turns a factory's capital lasts: a whole number, 1 or more

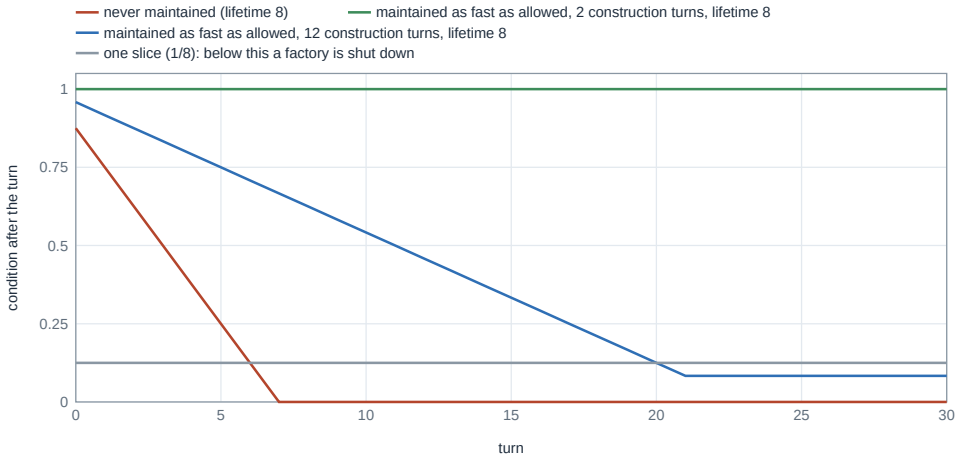
Things worth knowing:

- Every amount, and the capacity, is a number greater than 0. An empty table has to be written out as `{}`: leaving `inputs` or `construction` out is an error, so an empty one says you meant it.
- **Inputs are fixed proportions.** Making 12 units takes 12 times each input, with no substitution between inputs. A technology with no inputs makes its output from nothing.
- The output good can't also be an input, but it can be part of the capital: a toolmaker made of tools is fine.
- **Nothing checks the economics.** The loader doesn't care whether a technology can pay for itself, whether its inputs are sold anywhere, or whether its capital goods are durable.
- Both TOML table forms work, as for a POP type's endowment.

What the numbers mean

- **Wear.** Every turn, whether or not it runs, a factory uses up `1/lifetime` of its capital, one *slice*. A factory runs only while it holds at least one whole slice, so every turn of output costs exactly one slice; below that it is **shut down** and makes nothing, and a leftover part of a slice is simply lost. Condition doesn't slow a factory down: it runs at full capacity right up to the turn it shuts down. A full factory that nobody maintains runs exactly `lifetime` turns.
- **Construction.** Building a factory takes `construction_turns` turns and pays an equal share of the capital goods and the construction goods each turn. (Building new factories is not something a business can decide yet; the numbers exist now because maintenance is defined by them.)
- **Maintenance is a partial rebuild.** Topping a factory up by some share of its capital costs that share of both the capital goods and the construction goods, and a factory can regain at most `1/construction_turns` of its capital in one turn. Bringing one from 95% to 100% costs 5% of each bundle.
- **If `construction_turns` is greater than `lifetime`**, a factory wears out faster than it can be maintained, and once it is exhausted maintenance can never bring it back, because it never regains a whole slice. That is allowed, not an error: it is a legitimate thing for a setting to say.
- **Order.** Each turn a business's factories run, then wear, then are maintained. Running before wear is what gives a full factory exactly `lifetime` turns; maintaining after wear is what lets one slice of maintenance a turn hold a factory at full condition. How this fits into the turn is under How businesses trade and produce.

A worked example with the sample's `farm` (capital 4 tools, construction 8 labor, 2 construction turns, lifetime 8): it loses 0.5 tools of capital every turn. Building one would take 2 turns at 2 tools and 4 labor each. Topping it up from 75% to 100% costs 1 tool and 2 labor, and can be done in one turn, because 25% is within the 50% a turn that 2 construction turns allow. The chart follows a factory's **condition** (the share of its capital it holds) over thirty turns under three regimes: never maintained, maintained as fast as allowed when construction is quicker than the lifetime, and maintained as fast as allowed when it is slower. The never-maintained farm has run eight times when its condition reaches 0 at the end of turn 7; the slow-to-build factory sinks below one slice, shuts down, and never gets back.




```toml
# Inline form: the sample's farm.
[[technology]]
name = "farm"
inputs = { unimproved_grain = 1, unskilled_labor = 0.5 }
output = "grain"
capacity = 12
capital = { tools = 4 }
construction = { unskilled_labor = 8 }
construction_turns = 2
lifetime = 8

# Sub-table form. A well makes water from nothing and costs nothing to build.
[[technology]]
name = "well"
output = "water"
capacity = 20
construction_turns = 1
lifetime = 10

[technology.inputs]          # empty: water from nothing

[technology.capital]
```

```
stone = 5

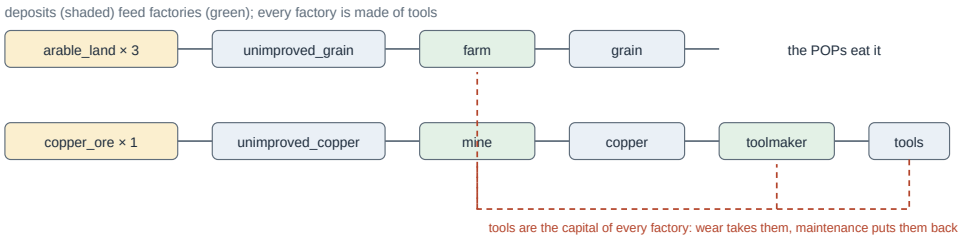
[technology.construction]    # empty
```

Errors, each with file, line and column: a missing key; a bad or duplicate name; an `inputs`, `capital` or `construction` that is not a table; an `output` that is not text; a `capacity` or an amount that is not a number or not greater than 0 (`inf` included); a `construction_turns` or `lifetime` that is not a whole number or is below 1; an empty `capital`; the output good among the inputs (`key 'grain' in inputs of technology must not be the output good`); an unknown good; an unknown key; and a `[[technology]]` in `start/`. Unknown-good errors are skipped while part of the setting could not be read.

The sample setting defines three technologies, one production chain:

Technology	Inputs per unit of output	Output	Capacity	Capital	Construction	Construction turns	Lifetime
`farm`	1 `unimproved_grain`, 0.5 `unskilled_labor`	`grain`	12	4 `tools`	8 `unskilled_labor`	2	8
`mine`	1 `unimproved_copper`, 1 `unskilled_labor`	`copper`	2.5	2 `tools`	4 `unskilled_labor`	2	8
`toolmaker`	2 `copper`, 1.5 `unskilled_labor`	`tools`	4	3 `tools`	6 `unskilled_labor`	3	24

The capacities match what the center's deposits produce each turn (12 unimproved grain and 2.5 unimproved copper). Every technology is made of tools and built with labor, so the toolmaker's output is what keeps every factory, its own included, in repair. At the center's prices (see Markets) a farm and a mine clearly pay their way and a toolmaker only just; the margin test under Businesses shows the numbers. The diagram shows the chain.



More detail: docs/technologies.md for the keys and every error message, docs/production.md for the exact rules of running, wear and maintenance, and docs/goods.md for the goods a technology names.

Businesses

A **business** is a named actor on one hex. It holds goods and cash, owns some of the deposits on its hex and the factories there, and is controlled by a **decider**: a program that chooses, every turn, what the business sells, buys, runs and maintains. Businesses are starting state, so they go in `start/` as `[[business]]` sections (in `rules/` it is an error). Six keys, all required:

Key	Meaning
`name`	Lowercase snake_case, unique among businesses. Factories name their owner, which is why businesses have names
`q`, `r`	A hex that exists on the map, as for deposits and POPs
`inventory`	`good name = amount` pairs: what the business holds at turn 0, each amount greater than 0. May be empty (`{}`) but must be written
`deposits`	`resource type = count` pairs: how many deposits of each type on its hex the business owns, each a whole number of 1 or more. May be empty (`{}`) but must be written
`decider`	The name of the decider that controls it. `arithmetic` is the only one so far

Things worth knowing:

- **Cash is just the hex's currency in the inventory.** There is no separate cash key, and a business that lists none starts with no cash. A business needs cash to buy what its deposits don't supply (labor, say) until its own sales come in: in the sample the farm starts with 100 silver against about 15 silver of purchases a turn.
- **One hex.** A business sits on one hex and can own only deposits and factories on that hex. A hex can hold several businesses, and a business can sit on a hex with no market; it then trades nothing, but still runs its factories from what it holds.
- **Ownership is written on the business**, not on the `[[deposit]]` entry, which doesn't change. Deposits of one type on one hex are identical, so it doesn't matter which units a business gets. The loader hands them out in order, with businesses taken in file order, so the same folder always gives the same owners. Claiming more deposits than the hex has left unowned is an error whose message says how many were available.
- **The decider name must be one the Python package knows.** The loader checks only that it is a valid name. When the setting is loaded in the developer UI or run with `check --turns`, one `python -m policyspace <name>` process is started for each distinct name, and a name the package doesn't know fails the load (see Checking a setting). Several businesses can share one decider; the sample's three all name `arithmetic`.
- Both TOML table forms work.
- In play, a business trades and runs its factories every turn; see How businesses trade and produce.

```toml
# Inline form: the sample's farm, which owns two of the center's three arable land deposits.
[[business]]
name = "center_farm"
q = 0
r = 0
inventory = { silver = 100 }
deposits = { arable_land = 2 }
decider = "arithmetic"

# Sub-table form. A smithy with some copper in stock and no deposits.
[[business]]
name = "smithy"
q = 1
r = 0
decider = "arithmetic"

[business.inventory]
silver = 40
copper = 5

[business.deposits]        # empty: owns no deposit
```

Errors, each with file, line and column: a missing key; a bad or duplicate name; a `decider` that is not text or not a valid name; bad coordinates or coordinates with no hex; an `inventory` or `deposits` that is not a table; an inventory amount that is not a number or not greater than 0; a deposit count that is not a whole number or is below 1; an unknown good in `inventory`; an unknown resource type in `deposits`; more deposits claimed than the hex has unowned (`key 'arable_land' in deposits of business claims more deposits than the hex has unowned (3 claimed, 1 available)`); an unknown key; and a `[[business]]` in `rules/`. The reference checks and the handing out of deposits are skipped while part of the setting could not be read. A misspelled decider name is not a loader error: it is caught when the deciders are started, by the window and by `check --turns`, not by plain `check`.

How deposits are handed out. Suppose a hex has three deposits of one type and two businesses on it, in file order, claim 2 and 1. The first gets two units, the second the third; a third business claiming one more gets the error above with `0 available`. File order decides who gets which unit, and since the units are identical it never matters.

How the arithmetic rule decides

Every turn each business is shown its circumstances (its hex's prices, its stockpile and cash, its factories with their technologies and condition, its deposits) and its decider answers with a **decision**: what to offer for sale, what to bid for, and for each factory the share of its capacity to run and the share of its capital to replenish. The only decider so far, `arithmetic`, plans each business from that view alone and remembers nothing between turns. For each kind of factory a business owns, it:

- checks whether **one turn at full capacity pays** at the hex's posted prices: what the output sells for, minus the inputs, minus the steady upkeep (one turn's share of the factory's capital and construction goods);
- if it pays, or breaks even, **runs the factory at capacity and maintains it** back to full condition (as far as one turn's maintenance allows, so a worn factory catches up over a few turns, and a shut-down one is revived slice by slice); if it doesn't pay, or if any good involved has no price on the hex, **leaves the factory idle and unmaintained**, so it slowly runs down;
- **sells everything it holds that the plan doesn't need**, last turn's output included, and **bids for what the plan needs that it doesn't hold**;
- if those purchases cost more than its cash plus what its sales will bring in, **scales every factory's run and maintenance down by one proportion** so that they fit;
- on a hex with no market, does nothing.

Every decision comes with a short written explanation of this arithmetic, its **trace**, which the developer UI shows in the business's section. The sample's farm at turn 0: grain sells at 2, so 12 grain bring 24; the inputs (12 unimproved grain at 0.5 and 6 labor at 1) cost 12; the upkeep (4 tools at 12 and 8 labor at 1, spread over the farm's 8-turn lifetime) costs 7 a turn. The margin is 5, so the farm runs. It holds 8 unimproved grain from its own fields, so it buys 4 more, plus 7 labor and half a tool, for 15 of its 100 silver:

```
farm: revenue 24; inputs 12; maintenance 7; margin 5: run and maintain
factory #0 (farm, condition 1): run 1, maintenance 0.125
budget 100 = cash 100 + offers 0; bids cost 15: within budget
```

The toolmaker is the opposite edge: its margin is 0.25 on a revenue of 48, so a small change in the price of copper or tools would idle it. The table below is computed from the sample's own files, so it changes if you edit them.

```
technology    revenue   inputs   upkeep   margin   verdict
farm               24       12        7        5   run and maintain
mine             12.5        5      3.5        4   run and maintain
toolmaker          48       46     1.75     0.25   run and maintain
```

The sample setting places three businesses, all on the center hex (`0, 0`) and all with decider `arithmetic`, one for each technology in the chain:

Business	Starting inventory	Owns	Factory
`center_farm`	100 `silver`	2 of the center's 3 `arable_land` deposits	a `farm`
`center_mine`	50 `silver`	the center's `copper_ore` deposit	a `mine`, starting worn
`center_toolmaker`	200 `silver`	no deposit: it buys its copper	a `toolmaker`

The center's third arable land deposit, and every deposit on other hexes, stay unowned. So each turn the center's 14.5 units of deposit output are split three ways: 8 unimproved grain to `center_farm`, 2.5 unimproved copper to `center_mine`, and 4 unimproved grain to the hex's stockpile of unowned goods, which is all the hex itself shows (see The developer UI).

More detail: docs/businesses.md for the keys, every error message, how deposits are handed out and the business stockpiles; docs/python.md for the exact arithmetic rule and how to read its trace; docs/decider.md for what a decider is shown and what it may answer; docs/decider-host.md for how the deciders are started and talked to; and docs/currency.md for why cash is just the hex's currency.

Factories

A **factory** is an asset on a hex that runs one technology and belongs to one business. Factories are starting state, so they go in `start/` as `[[factory]]` sections (in `rules/` it is an error). Four keys are required and two are optional:

Key	Meaning
`q`, `r`	A hex that exists on the map. For now it must be the owner's hex
`technology`	The name of a technology from `rules/`
`owner`	The name of a business. Every factory has exactly one owner
`condition`	Optional, default 1. How much of its technology's capital the factory holds, as a share from just above 0 up to 1
`count`	Optional, 1 to 1,000,000, default 1. Places that many identical factories at once, the same rule as a deposit's `count`

- **Factories have no names.** Each gets a number in load order (file order, then entry order, with an entry's `count` copies numbered consecutively), so the same folder always numbers them the same way. The deciders and the developer UI refer to a factory by that number.
- **Condition is one number, not a table of goods.** Wear and maintenance always change a factory's capital as a whole, in the technology's own proportions, so the share is all there is to store. The goods in a factory are the technology's capital times the condition: the sample's mine at 0.75 holds 1.5 of its 2 tools. A condition of 1 is a factory in full repair; a factory can't be placed at 0. **The condition changes during play**: wear takes a slice every turn and maintenance puts back what the business pays for, so a fresh world's factories are already past turn 0's wear (see Running a setting). A factory nobody maintains wears out in `lifetime` turns.
- A hex or a business may have any number of factories, of any technologies. Nothing checks that a factory makes economic sense: whether its owner can get its inputs or sell its output is the owner's problem.
- The factory must be on its owner's hex. This restriction goes once trade between hexes

exists.

```
# The sample's mine: three quarters of its capital left.
[[factory]]
q = 0
r = 0
technology = "mine"
owner = "center_mine"
condition = 0.75

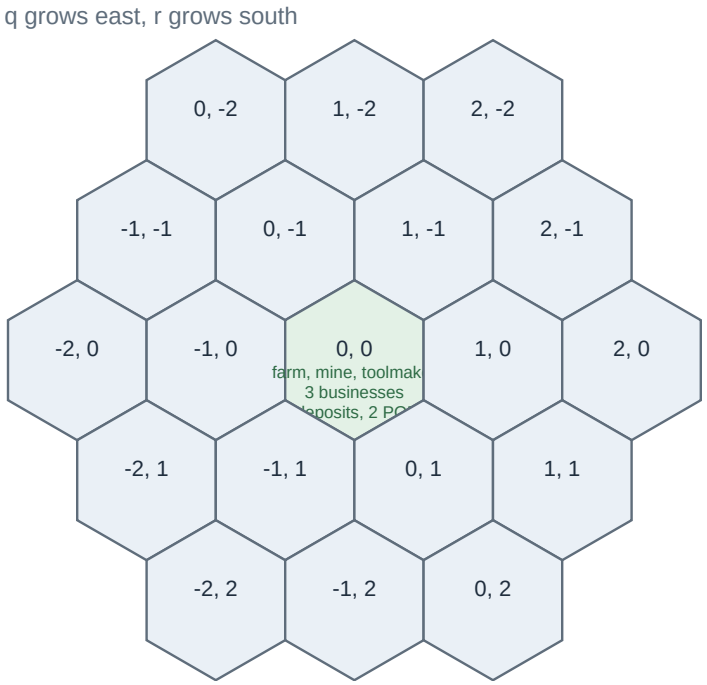
# Two identical farms in full condition.
[[factory]]
q = 0
r = 0
technology = "farm"
owner = "center_farm"
count = 2
```

Errors, each with file, line and column: a missing `q`, `r`, `technology` or `owner`; bad coordinates or coordinates with no hex; a `technology` or `owner` that is not text; an unknown technology or business; a factory that is not on its owner's hex (`factory at (1, 0) is not on the hex of its owner 'farm' at (0, 0)`); a `condition` that is not a number, is 0 or below, above 1, infinite or not a number; a `count` that is not whole or is out of range; an unknown key; and a `[[factory]]` in `rules/`. The reference checks, the owner's-hex check included, are skipped while part of the setting could not be read.

The sample setting places one factory per business on the center hex, each running the technology its business is named for:

Number	Technology	Owner	Condition at turn 0	Capital held
0	`farm`	`center_farm`	1	4 `tools`
1	`mine`	`center_mine`	0.75	1.5 of 2 `tools`: six turns of its eight-turn lifetime
2	`toolmaker`	`center_toolmaker`	1	3 `tools`

The mine starts worn so that the sample has a factory to repair: under the arithmetic rule it is maintained back to full in turn 0 (see How businesses trade and produce). The diagram shows the center hex with its deposits, POPs, businesses and factories.



More detail: docs/factories.md for the keys, every error message, condition and count; docs/technologies.md for the capital bundle the condition is a share of; and docs/production.md for wear and maintenance.

Running a setting

A loaded setting becomes a running **world**. The rules stay fixed; the world holds the clock (starting at turn 0), a stockpile of unowned goods on every hex, for every POP its current size and its own stockpile, for every business its own stockpile (filled from its inventory before turn 0), and every factory's condition. Each turn has three parts:

1. **Emit and endow.** At the start of the turn every deposit produces its resource type's `output` of its good: into its owner's stockpile if a business owns it, otherwise into the hex's stockpile of unowned goods. Then every POP receives its type's endowment times its size, into its own stockpile.
2. **Decide, trade, produce and grow.** First every business's decider is asked what the business should do this turn. Then, on every hex, everyone sells before anyone buys: each POP sells the market what it does not mean to keep and each business what its decision offers; then each POP buys what it wants with the proceeds and each business what its decision bids for, as far as its cash allows. Each POP consumes. Each business then runs, wears and maintains its factories. Finally every POP's size changes by its growth rate. The rest of this section explains it.
3. **Perish.** At the end of the turn, goods that have reached their limit are removed from every stockpile: the hexes', the POPs' and the businesses'.

The world is observed after part 2 and before part 3, so what you see at any turn is the world after that turn's decisions, trading, production and growth, with nothing perished yet. That includes turn 0: a freshly loaded world has already run turn 0, which is why its POPs are not at the sizes written in the setting and its factories have already run (or, for an idle business, worn) once (see Turn reports below).

`perishes_after` takes effect in part 3. With 1, output is usable during the turn it appears and gone when it ends. With 3, it is gone at the end of the third turn it exists. Goods without `perishes_after` last forever and accumulate. (`movable` still has no effect.)

A **stockpile** holds the total of each good, and for each total, how much perishes at the end of which turn. For example, a good with `perishes_after = 3` produced at 5 per turn:

Turn	Held during the turn (after the emit)	Removed when the turn ends
0	5	0
1	10	0
2	15	5 (the batch from turn 0)
3	15	5 (the batch from turn 1)
4	15	5 (the batch from turn 2)

How POPs trade

Each turn, once the endowment has arrived, the market on the POP's hex works out what the POP wants to end up with and makes the trades. In plain terms:

- **Income is everything the POP holds**, valued at the hex's prices: this turn's endowment, plus any durable goods it still has, plus any currency left over from earlier turns.
- **Subsistence comes first.** The POP first covers every utility good's subsistence amount, times its size. Whatever income is left is split between its utility goods by their shares. Goods the hex does not price are left out of this sum: a held one is simply kept, and a wanted one cannot be bought.
- **Below subsistence, everything is scaled down.** If the income cannot cover the subsistence bundle, the POP spends all of it and gets the same fraction of every subsistence amount, leisure included: a peasant facing dear grain sells more of its labor than it would like to keep and still goes short of grain.
- **Sales first, then purchases.** On each hex every sale, by POPs and businesses alike, happens before any purchase. With today's fixed-price market this changes nothing; it is there so that the later live market, which can run short, is fair.
- **Then it consumes** the goods in its utility, up to what it wanted. The labor it kept is its leisure, consumed the same way. Goods outside its utility are never consumed: they are sold if the hex prices them and kept if it does not.
- **Leftover currency stays.** A POP spends essentially all its income; floating-point rounding can leave a tiny sliver of currency, which stays in its stockpile and counts as income next

turn. Nothing is rounded.
- **No market, no trade.** A POP on a hex with no market keeps what it has and consumes what it can. The sample's north-east peasant consumes all its labor as leisure and goes without grain every turn.

Some consequences for setting authors: a durable good a POP is endowed with but neither wants nor can sell piles up in its stockpile; if none of the goods a type gives a share to is priced on its hex, its spare income is kept as currency and, for a durable currency, piles up too; and a POP that cannot afford its subsistence falls short on everything in the same proportion.

A worked example from the sample at turn 0, at a typical hex's prices (labor 1, grain 2 and copper 5 silver):

	Center peasant, size 60	**North-east peasant, size 12 (no market)**
Receives	600 labor	120 labor
Income	600 silver	nothing can be sold, so 0
Subsistence needs	120 labor and 120 grain, costing 360 silver	24 labor and 24 grain, but grain cannot be bought
Left over	240 silver, split 0.2 to labor and 0.8 to grain	nothing
Wants to end up with	168 labor and 216 grain	the 120 labor it holds, and 0 grain
Sells	432 labor for 432 silver	nothing
Buys	216 grain for 432 silver	nothing
Consumes	168 labor and 216 grain	120 labor
Holds afterwards	at most a rounding sliver of silver	nothing

Per person: a peasant sells 7.2 of its 10 labor, buys 3.6 grain and keeps 2.8 labor as leisure, and an artisan at the center sells 8.5 labor and buys 3 grain and 0.5 copper. The chart shows how a peasant's choices change with the price of grain. Above a price of 4 (with labor at 1) its 10 labor can no longer cover its subsistence, and everything scales down together.



How businesses trade and produce

Each turn, after the deposits have produced and the POPs have been endowed, every business's decider is asked for a **decision**: what to offer for sale, what to bid for, and for each factory the share of its capacity to run and the share of its capital to replenish (see How the arithmetic rule decides). The engine checks the decision against what the business actually has: a factory it doesn't own, or a good its hex doesn't price, gets the whole decision **refused**, and the business does nothing that turn. A valid decision is then acted on:

- **The business sells its offers and buys its bids on its hex's market, in the same round as the POPs.** Every sale on the hex happens before any purchase. A business sells what it offered, or everything it holds of that good if it offered more.
- **Purchases are capped by cash.** It buys what it bid for at the posted prices, unless that costs more than the cash it has after its sales. Then **every purchase is cut by the same proportion**: a business with 10 silver bidding for goods worth 50 buys a fifth of each, so it ends up with a smaller set of goods in the proportions it asked for, rather than all of one good and none of another.
- **Then its factories run, wear and are maintained**, in that order, from the business's own stockpile. A factory holding less than one slice makes nothing. If the business is short of an input or a maintenance good (because a cash cut left it short, say), the factories that need it run, or are maintained, less, in proportion to the shortage, and a factory short of two goods is cut by the worse one. The engine never changes the fractions the decider asked for; the shortage does, and both numbers are recorded, so a cut always shows as "asked" above "done".
- **Output is sold the next turn.** What factories make goes into the business's stockpile and is offered at the next turn's decision. So at turn 0 no business has anything of its own to sell, and from turn 1 on each sells last turn's output. Labor a business buys is used the same turn; anything left of it perishes at the end of the turn like any labor.
- **A business that decides nothing does nothing, but its factories still wear.** That covers a business whose decision was refused, one whose decider failed this turn (see Checking

a setting), and every business when nothing is running the deciders at all. Its factories lose one slice every turn and eventually shut down.
- **A business on a hex with no market** trades nothing and runs its factories from what it holds: its deposits' output and its starting inventory.

The sample's three businesses under the arithmetic rule, at the center's prices (unimproved grain 0.5, labor 1, grain 2, copper 5, tools 12 silver):

	`center_farm`	`center_mine`	`center_toolmaker`
Holds at turn 0's decision	8 unimproved grain, 100 silver	2.5 unimproved copper, 50 silver	200 silver
Sells at turn 0	nothing	nothing	nothing
Buys at turn 0	4 unimproved grain, 7 labor, 0.5 tools	4 labor, 0.75 tools	8 copper, 6.25 labor, 0.125 tools
Its factory at turn 0	runs at capacity: 12 grain from 12 unimproved grain and 6 labor; wears 1/8; maintained by 1/8, back to 1	runs at capacity: 2.5 copper; wears 1/8; maintained by 3/8, from 0.75 to 1	runs at capacity: 4 tools from 8 copper and 6 labor; wears 1/24; maintained by 1/24, back to 1
Cash after turn 0	85 silver	37 silver	152.25 silver
Sells at turn 1	12 grain	2.5 copper	3.875 tools (it keeps 0.125 for its own upkeep)
Cash after turn 1	94 silver	43.5 silver	152.5 silver

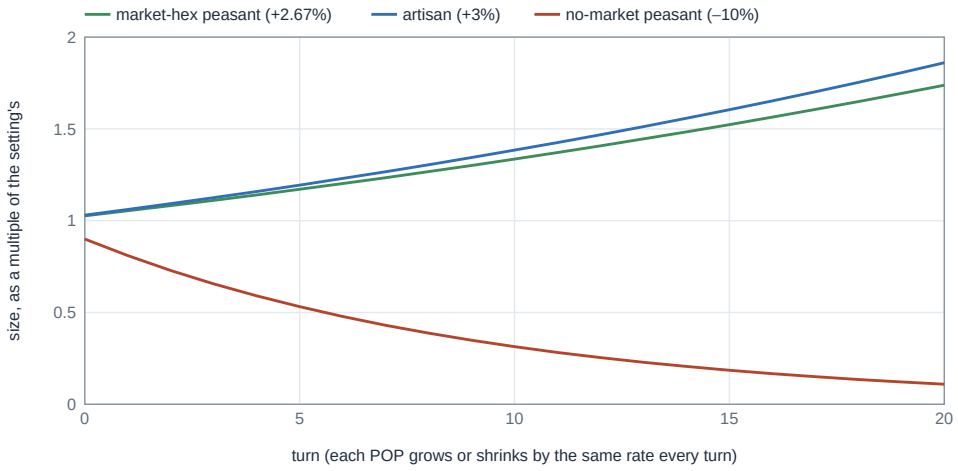
From turn 1 on every turn repeats, each business gets a little richer, and every factory stays at full condition. The developer UI shows these numbers when you select the center hex (see The developer UI).

A worked example of the cash cut. A business with 10 silver bids for 10 labor (at 1 silver) and 4 tools (at 10), costing 50. Every bid is cut to a fifth: it buys 2 labor and 0.8 tools, spending all 10 silver. Its smelter, asked to run at full capacity, finds a fifth of the labor it needs and runs at a fifth, while the record still shows the full run it asked for. The arithmetic rule scales its own plan down to its budget before bidding, so in the sample this cut never happens; it is a safety net under any decider that asks for too much.

More detail: docs/trade.md for the round, the cash cap and when the factories run; docs/production.md for running, wear and maintenance; docs/decider.md for the decision, its checks and why asking for too much is allowed; and docs/decider-host.md for how the deciders are asked.

How a POP's size changes

After trading and consuming, each POP's size is multiplied by one plus its growth rate (the rule and the type's parameters are under How POPs grow). In the sample a peasant on a market hex has 240 silver to spare against a subsistence cost of 360, so its index is 2/3 and its rate about +2.67% a turn; the center artisan has an index of 1 and grows 3%; and the north-east peasant, which cannot get grain at all, has an index of −1 and shrinks 10% a turn: 12 people become 10.8 at turn 0, 9.72 after one step and 8.748 after two. The rate depends only on prices and the type, not on size, so the sample's sizes compound at the same rate every turn, and the next turn's endowment, trade and consumption grow or shrink with them. The chart shows twenty turns of this, each POP relative to the size written in the setting (turn 0 already includes turn 0's growth).



Turn reports

Each turn produces a report with totals over the whole map: per good, what `perished` when the previous turn ended, what deposits `emitted`, what POPs were `endowed`, and what POPs `sold`, `bought` and `consumed`; then the total `population` after growth; then four business totals, kept apart from the POPs' so that the POP numbers keep their meaning: what `businesses sold` and `businesses bought`, what their factories `produced`, and what the factories `used` (inputs and maintenance goods together). Each list is in the order the goods are defined (so the sample's `unskilled_labor` comes first, `copper` before `grain`, and `tools` last), lists only goods with an amount, and prints `nothing` when empty. If a decider failed during the turn, a line saying so follows the turn's line (see Checking a setting). The sample over turns 0 to 2, as the checker prints it (the next section gives the command; it starts the sample's Python decider, so it needs the environment from Project overview):

```
Loaded setting 'Sample Setting' (format version 1): 7 goods, 19 hexes, 2 resources,
10 deposits, 18 markets, 2 pop types, 16 pops, 3 technologies, 3 businesses, 3
factories
Turn 0: perished nothing; emitted 10 unimproved_copper, 24 unimproved_grain; endowed
4100 unskilled_labor; sold 2878.6 unskilled_labor; bought 5 copper, 1426.8 grain;
```

```
consumed 1221.4 unskilled_labor, 5 copper, 1426.8 grain; population
419.4466666666666; businesses sold nothing; businesses bought 17.25 unskilled_labor,
4 unimproved_grain, 8 copper, 1.375 tools; produced 2.5 copper, 12 grain, 4 tools;
used 17.25 unskilled_labor, 2.5 unimproved_copper, 12 unimproved_grain, 8 copper,
1.375 tools
Turn 1: perished 0.00000000000000022220446049250313 unskilled_labor, 7.5
unimproved_copper, 16 unimproved_grain; emitted 10 unimproved_copper, 24
unimproved_grain; endowed 4194.466666666667 unskilled_labor; sold 2955.6459999999997
unskilled_labor; bought 5.15 copper, 1464.9479999999999 grain; consumed
1238.8206666666667 unskilled_labor, 5.15 copper, 1464.9479999999999 grain;
population 429.2982444444444; businesses sold 2.5 copper, 12 grain, 3.875 tools;
businesses bought 16.25 unskilled_labor, 4 unimproved_grain, 8 copper, 0.75 tools;
produced 2.5 copper, 12 grain, 4 tools; used 16.25 unskilled_labor, 2.5
unimproved_copper, 12 unimproved_grain, 8 copper, 0.8749999999999999 tools
Turn 2: perished 0.00000000000000022220446049250313 unskilled_labor, 7.5
unimproved_copper, 16 unimproved_grain; emitted 10 unimproved_copper, 24
unimproved_grain; endowed 4292.982444444444 unskilled_labor; sold 3034.7550599999995
unskilled_labor; bought 5.304500000000001 copper, 1504.1162799999997 grain; consumed
1258.2273844444444 unskilled_labor, 5.304500000000001 copper, 1504.1162799999997
grain; population 439.55036096296294; businesses sold 2.5 copper, 12 grain, 3.875
tools; businesses bought 16.25 unskilled_labor, 4 unimproved_grain, 8 copper, 0.75
tools; produced 2.5 copper, 12 grain, 4 tools; used 16.25 unskilled_labor, 2.5
unimproved_copper, 12 unimproved_grain, 8 copper, 0.8749999999999999 tools
```

Where the numbers come from. **The POPs:** the setting's 410 people receive 10 labor each, 4100 in all. Each of the 388 peasants on a market hex sells 7.2 labor and buys 3.6 grain, the 10 artisans sell 8.5 labor each and buy grain and copper, and the 4100 − 2878.6 = 1221.4 labor not sold is consumed as leisure (the no-market peasant's 120 included). The population at turn 0 is already about 419.4, not 410, because turn 0's growth has run, and each later turn's `endowed` is 10 labor times the previous turn's population. **The businesses:** at turn 0 they buy 17.25 labor, 4 unimproved grain, 8 copper and 1.375 tools and their factories make 2.5 copper, 12 grain and 4 tools, but they sell nothing, because nothing has been made yet. From turn 1 they sell last turn's output, 2.5 copper, 12 grain and 3.875 tools, buy a little less (every factory is now in full condition, so the mine's catch-up maintenance is over), and the business lines repeat every turn. **Perishing:** the POPs' labor never appears in `perished`, because it is all sold or consumed during the turn, and from turn 1 `perished` holds only the *unowned* deposits' output (7.5 unimproved copper and 16 unimproved grain), because the factories use what the owned deposits produce. The `0.00000000000000022220446049250313 unskilled_labor` in front of it is a floating-point sliver: the arithmetic rule's labor bid for the toolmaker is a hair above what its maintenance uses, and the leftover perishes. The engine rounds nothing, which is also why amounts grow long tails of digits after turn 0 and why turn 1 says `0.8749999999999999 tools` rather than 0.875.

More detail: docs/world.md for the turn and the report, docs/trade.md for the trade round, docs/demand.md for how what a POP wants is worked out, docs/production.md for the factories, and docs/growth.md for growth.

Checking a setting

The checker loads a setting and reports every problem in one go, with file, line and column. It never produces a half-loaded setting.

```
dev-ui check <folder> [--turns N]
```

From the project root, that is `cargo run -p policyspace-dev-ui -- check settings/sample`. On success it prints one line:

```
Loaded setting 'Sample Setting' (format version 1): 7 goods, 19 hexes, 2
    resources, 10 deposits, 18 markets, 2 pop types, 16 pops, 3
    technologies, 3 businesses, 3 factories
```

(Counts use singular wording where it applies: `1 hex`, `1 resource`, `1 deposit`, `1 market`, `1 pop type`, `1 pop`, `1 technology`, `1 business`, `1 factory`. Currencies are not counted, since every hex has exactly one.) With `--turns N` (N from 0 to 10,000) it then starts the setting's deciders, creates the world and prints one report line per turn, from turn 0 through turn N, as shown in Turn reports; `--turns 0` prints only turn 0.

Problems look like this:

```
rules/a.toml:3:18: key 'perishes_after' in good must be at least 1
rules/a.toml:6:6: duplicate good name 'copper' (first defined at
    rules/a.toml:2:6)
rules/a.toml:10:10: unknown good 'gold'
rules/p.toml:14:1: utility of pop_type needs at least one good with a
    positive share
rules/p.toml:20:11: key 'decline' in growth of pop_type must be a finite
    number of at least 0 and less than 1
rules/t.toml:4:12: key 'grain' in inputs of technology must not be the
    output good
start/b.toml:6:14: key 'arable_land' in deposits of business claims more
    deposits than the hex has unowned (3 claimed, 1 available)
start/f.toml:1:1: factory at (1, 0) is not on the hex of its owner 'farm'
    at (0, 0)
start/k.toml:5:3: 'coin' is the currency at (0, 0), so it can't have a
    price
start/m.toml:4:1: unknown key 'terrain' in hex
start/m.toml:6:1: duplicate hex at (0, 0) (first defined at
    start/m.toml:1:1)
start/m.toml:9:1: missing required key 'currency' in hex
start/p.toml:7:1: duplicate pop of 'peasant' at (0, 0) (first defined at
    start/p.toml:1:1)
13 error(s) found in my_setting
```

If part of the setting could not be read (a missing folder, or a broken or unreadable file), errors that refer to names or hexes, such as `unknown good`, are not shown; fix the structural problem

first.

Deciders that won't start. With `--turns`, the checker starts one Python process per decider name the setting's businesses carry (none for a setting without businesses). If one can't start, the checker prints the reason and `1 error(s) found in <folder>`, runs no turns, and exits with code 1. The likeliest cause for a setting author is a misspelled `decider` name, which the Python package doesn't know:

```
decider 'rule': '...  .venv  Scripts  python.exe -m policyspace rule' exited
    (exit code: 2) as soon as it started
its error output ends with:
  error: unknown decider 'rule' (known: arithmetic)
1 error(s) found in my_setting
```

The other cause is a missing Python environment; that message names the path where it was expected and the two setup commands. **Plain `check <folder>`, without `--turns`, never starts the deciders, so it can't catch a misspelled decider name; `--turns 0` does.**

Deciders that fail during a turn don't stop the world. If a decider's process crashes, takes too long to answer (more than 5 seconds, unless `POLICYSPACE_DECIDER_TIME_LIMIT` says otherwise), or answers something that isn't a valid reply, the businesses it controls do nothing that turn (their factories still wear), the turn goes on, and a line follows that turn's report, then the last lines the process wrote to its error output:

```
Turn 3: decider 'arithmetic' failed: the process exited (exit code: 1)
    before answering
    stderr: Traceback (most recent call last):
    stderr: ...
```

The process is started again at the next turn; if it keeps failing, a line appears every turn. A failure during the turns doesn't change the checker's exit code. A single business whose decision breaks the rules (a factory it doesn't own, a good with no price) is not a failure line: that business just does nothing that turn, and the developer UI shows why. Printing to standard error (`print(..., file=sys.stderr)` in Python) is the way to debug a decider, since those lines are what the failure line quotes.

Exit code	Meaning
0	The setting loaded (and, with `--turns`, its deciders started)
1	The setting has errors, or its deciders couldn't start
2	Wrong usage (a three-line usage message is printed)

The checker never opens a window, so it is the form to use in scripts. `dev-ui --version` prints the engine's name and version. More detail: docs/setting-loader.md, docs/world.md for `--turns`, and docs/decider-host.md for how deciders are started and what counts as a

failure.

The developer UI

The developer UI is a window that shows a setting's map and lets you step through turns and inspect the world: each hex's goods, prices, POPs and businesses, what every POP did this turn, what every business decided and what came of it, and each factory's condition and output.

- `dev-ui <folder>` opens the window with a setting loaded.
- `dev-ui` alone opens it with an **Open setting…** button (a system folder picker; cancelling changes nothing).

For example: `cargo run -p policyspace-dev-ui -- settings/sample`. Loading a setting that has businesses starts its deciders as Python processes (one per decider name) and keeps them running while the setting is loaded; Reload, opening another folder and closing the window end them. A decider that can't start is shown like a setting error (see Checking a setting).

Reading the map

Each hex is a pointy-top hexagon, rows run east-west, `q` increases eastward and `r` increases southward, exactly as in The map. The sample setting shows as a hexagon of 19 hexes. When hexes are drawn as hexagons, three small marks say what a hex holds, so you can see where to click: a hex with at least one deposit shows a **filled circle** below its label, a hex with at least one POP a **hollow circle** above it, and a hex with at least one business a **filled square** at the lower left. A hex can show any combination; the sample's center shows all three.

Controls

- Drag to pan; scroll to zoom toward the cursor; **Fit** shows the whole map again.
- When zoomed in far enough, each hex shows its coordinates as `q, r` (the center of the sample is `0, 0`).
- Hovering highlights a hex and shows its coordinates in the status bar.
- The status bar also shows the setting name, the hex count and the current turn.

Unusual maps are handled: an empty map shows "This setting has no hexes."; holes, islands and negative coordinates draw correctly; for maps spanning huge distances, hexes too small to see are drawn as dots (with no marks), and zooming in turns them back into hexagons.

Inspecting a hex

Click a hex to select it (a thick accent-colored outline, distinct from the hover highlight). When hexes are drawn as dots, a click near a dot selects it. Click empty space or press Esc to clear. Dragging to pan never changes the selection.

The **hex panel** on the right shows, for the selected hex:

- its coordinates;
- its deposits grouped by type, for example `arable_land × 3: 4 unimproved_grain each, 12 total`. The lines list every deposit on the hex, owned or not;
- its stockpile of **unowned goods**: each good's total, and the parts of it with a countdown to when they perish: `perishes this turn`, `perishes in N turns`, or `never perishes`. Output from deposits a business owns is in that business's stockpile instead, so on the sample's center the stockpile shows only `4 unimproved_grain`, although its deposit lines add up to 14.5;
- its **market**: the hex's currency (`Currency: silver`), then each posted price as the amount of currency one unit costs (`grain: 2 silver`), in the order the goods are defined. A hex with a market but no prices says `No prices posted.`, and a hex with no market says `No market.`;
- its **POPs**: one collapsible section per POP, open at first, headed by its type and current size (`peasant × 61.6`), or `No POPs.`;
- its **businesses**: one collapsible section per business, open at first, headed by its name and current cash (`center_farm: 85 silver`), or `No businesses.`

Reading the countdown: suppose hex `(0, 0)` holds 15 of a good that lasts 3 turns, at turn 2, with 5 produced each turn. The panel lists 5 that perish this turn, 5 that perish in 1 turn and 5 that perish in 2 turns: the oldest batch is about to go, the newest has two more turns to live.

What "this turn" means. Between steps the world has just run a turn: the businesses have decided, everyone has traded, the POPs have consumed and grown, the factories have run, worn and been maintained, and nothing has perished yet (see Running a setting). So the panel shows the outcome of the turn in the status bar: each POP's size after that turn's growth and its stockpile after consumption, each business's decision at that turn's decision point and the trade and production it led to, each factory's condition after that turn's wear and maintenance, and each business's stockpile after its factories have run. Each step replaces it all with the next turn's figures. A POP or business section you have collapsed stays collapsed when you step.

A POP section, for the sample's center peasant at turn 0, looks like this:

```
peasant × 61.6
  Started at 60; index 0.6667; rate +2.67%
  Income 600 silver; subsistence cost 360 silver; supernumerary 240 silver

  Good              Held   Demanded   Subsistence   Sold   Bought
                    Consumed
  unskilled_labor    600        168           120    432        0
                     168
  grain                0        216           120      0      216
                       216
```

```
  Carried over: 0 silver
  Stockpile: Empty.
```

- The **heading** is the type and the size after this turn's growth.
- The **growth line** gives the size the turn began with, the subsistence index and the growth rate (see How POPs grow). 60 peasants with 240 silver to spare against a subsistence cost of 360 have an index of 240 / 360 = 0.6667, a rate of +2.67%, and so become 61.6. Hovering the line shows the index and rate unrounded.
- The **demand line** is the POP's income, what its subsistence bundle costs at the hex's prices, and the difference, all in the hex's currency: 600 labor at 1 silver, against 120 labor at 1 and 120 grain at 2.
- The **goods table** has one row per good involved: what the POP held when it came to market, what it wanted to end up with, what it needs to subsist (its type's amount times its starting size), and what it sold, bought and consumed. Here the 240 spare silver went 0.2 to labor and 0.8 to grain, so the POP kept 168 labor and bought 216 grain. **A row in red means the POP wanted less of that good than it needs to subsist**, so a struggling POP and the good it lacks stand out.
- **Carried over** is the currency left after consumption: usually 0 or a tiny rounding sliver.
- The **stockpile** is the POP's own, written exactly like the hex's.

The north-east peasant on `(2, -2)`, whose hex shows `No market.`, makes the contrast: its section reads `peasant × 10.8`, `Started at 12; index -1.0000; rate -10.00%`, an income of 0, and its grain row in red (demanded 0 against a subsistence need of 24), while all 120 of its labor is held, demanded and consumed as leisure.

A business section, for the sample's `center_farm` at turn 0, looks like this:

```
center_farm: 85 silver
  Decider: arithmetic
  Answer: valid decision
  Deposits: arable_land × 2: 4 unimproved_grain each, 8 total
  Cash: 100 silver before; 100 after sales; 85 after purchases;
      proportion 1

  Trade
  Good               Offered   Sold   Bid   Bought
  unskilled_labor          0      0     7        7
  unimproved_grain         0      0     4        4
  tools                    0      0   0.5      0.5

  Factories
  #   Technology   Condition before   Condition after   Remaining life
      Run asked   Run done   Output     Wear    Maintenance asked
      Maintenance done
  0   farm                       1                1          8 turns
```

```
    1          1   12 grain   0.125              0.125
    0.125

Trace                                (collapsed; click to open)
Stockpile: 12 grain (12: perishes in 2 turns); 85 silver (85: never
    perishes)
```

- The **heading** is the business's name and its cash after this turn's trading: the hex's currency in its stockpile.
- **Decider and answer:** the decider's name, and whether this turn's decision was `valid decision`, `refused, N error(s)` (each error follows, in red, with the place it was found, for example a factory number the business doesn't own or a good the hex doesn't price), or `nothing (no decision was sent, or its decider failed)`.
- **Deposits** the business owns, written like the hex's deposit lines, or `Owns no deposits.`
- **Cash** through the round: before trading, after its sales and after its purchases, and the **proportion** the engine cut its purchases by. 1 means it could afford everything it bid for; a value below 1 is in red and means it asked for more than it could pay for, so it got that share of each good and its factories ran less (see How businesses trade and produce).
- The **trade table** has one row per good, with what the business offered beside what it sold and what it bid beside what it bought, so a cut shows as "asked" above "done" in one row. A refused decision shows what was asked with everything done at 0; a business that decided nothing shows `Nothing was traded.`
- The **factories table** has one row per factory: its number (the one the decider uses), its technology, its condition before and after this turn, its **remaining life** (how many more turns it can run with no maintenance, or `shut down` in red), the run asked and done, the output it made, the wear it took, and the maintenance asked and done. Hovering shows the inputs the run used (on Output), what the maintenance cost (on Maintenance done), and the capital the factory now holds (on Condition after). Here the farm bought what its own fields didn't cover, ran at full capacity, lost an eighth of its condition to wear and paid it back with maintenance, so it is back at 1 with 8 turns of life. The hover texts read `Inputs: 6 unskilled_labor, 12 unimproved_grain`, `Maintenance cost: 1 unskilled_labor, 0.5 tools` and `Capital: 4 tools`.
- **Trace** is the decider's written explanation of its arithmetic (see How the arithmetic rule decides), collapsed at first. Open it to see why the business did what it did.
- The **stockpile** is the business's own, written exactly like the hex's and the POPs'. It holds this turn's output, waiting to be sold next turn, and the business's cash. After one Step the farm has sold the 12 grain and the heading reads `center_farm: 94 silver`.

The contrast is a business that decided nothing, for example the same farm while its decider is failing: `Answer: nothing (no decision was sent, or its decider failed)`, `Nothing was traded.`, and a factory row with run 0, output `0 grain`, condition 1 → 0.875 and `7 turns` of life, because an idle factory still wears.

Sizes and amounts are written in full, with long decimals after a few turns, as in the turn log,

and nothing in a business section is rounded either: the sample's toolmaker is shown buying `0.12499999999999989` tools, exactly as its decider asked. Only the index (4 decimals), the rate (a percentage to 2 decimals) and the utility shares (up to 4 decimals) are rounded, and hovering shows the full values.

Other messages: "Click a hex to inspect it.", "No deposits.", "Empty.", "No market.", "No prices posted.", "No POPs.", "No businesses.", "Owns no deposits.", "Nothing was traded.", "No factories." and "This setting has no hexes."

Stepping turns

The **Step** and **Step x10** buttons in the top bar advance the world one or ten turns (shortcuts: Space and Shift+Space). They work whenever the setting loaded without errors, even one with no hexes. The status bar's turn follows. The world starts at turn 0 on every load.

The world panel

The **world panel** on the left has six sections, which collapse; the panels can be resized.

- **Turn log:** every turn report since the load, turn 0 included, newest first, in the same wording as `check --turns` (see Turn reports), the businesses' totals included. When a decider fails during a turn, its failure line and the last lines of its error output follow that turn's line, exactly as the checker prints them.
- **Goods:** each good's name, how long it lasts, whether it is movable or fixed, and three amounts: **In hexes**, the unowned deposit output held in hex stockpiles; **In POPs**, what the POPs own; and **In businesses**, what the businesses own (their deposits' output, their cash, what they bought and their factories' output). In the sample the raw goods appear under In hexes (the unowned deposits) and In businesses (the owned ones), silver under In POPs and In businesses, and tools only under In businesses.
- **Resource types:** each type's name, the good it produces, its output per deposit and its deposit count.
- **POP types:** each type's name, its endowment (or `nothing`), its utility goods with their subsistence amounts and shares as the engine uses them (hover for the full value), its growth parameters, the number of POPs of the type and their total size. The total sizes add up to the turn log's `population`.
- **Technologies:** each technology's recipe (inputs per unit of output, or `nothing`), output, capacity, capital, construction goods and turns, lifetime, and how many factories run it.
- **Businesses:** each business's hex, decider, cash, number of factories, and this turn's answer: `valid`, `refused` or `nothing`. A refused business's row is in red, so a broken decider shows without clicking around the map.

The author's loop

1. Edit a setting file (for example, change some prices, deposits, POPs or businesses), or a decider in `python/policyspace/`.

2. Click **Reload**. The world returns to turn 0, with a log of one line, and the deciders are started afresh, so an edited decider takes effect. Your pan and zoom stay, and so does your selection if the reloaded map still has a hex at those coordinates.
3. Step a few turns and check the stockpiles, prices, POPs, and the businesses' trades and factories. To see why a business did something, open its Trace.

If the setting has errors, the window shows the full error list (the same lines the checker prints) and no map or side panels, and the Step buttons are disabled. A decider that won't start (a misspelled `decider` name, or a missing Python environment) shows the same way, with the message from Checking a setting. Fix the files and Reload again. A failed Reload doesn't lose the selection. A broken folder given on the command line opens the window showing its errors. Opening a different folder clears the selection and the log.

Not available yet

Auto-run, stepping by another number, coloring hexes by data (population included), a POP's or a business's history over turns or charts of it, editing the world, selecting several hexes, and taking over a business to send its decisions by hand from the window (planned for later).

More detail: docs/dev-ui.md for the panels and marks, with docs/dev-ui.md#businesses-factories-and-technologies for the business sections and tables; docs/world.md for turns and stockpiles; docs/market.md and docs/currency.md for the Market section; docs/demand.md, docs/trade.md and docs/growth.md for the numbers in a POP section; and docs/trade.md, docs/production.md, docs/decider.md and docs/python.md for the numbers, errors and trace in a business section.

Technical documentation

For developers who want to understand or modify the project, see the technical documentation.